

Finding BOOM CPU Bugs with an AI Loop and Two Reference Models

Divya Kohli

University of California, Santa Cruz

dkohli1@ucsc.edu

Abstract—We built a tool that looks for bugs in the BOOM CPU. BOOM is an out-of-order RISC-V core. The tool runs a test program on BOOM and also on reference models that follow the RISC-V rules, then compares the results. If BOOM does something different, that may be a bug. The tool has four parts. The first part reads BOOM’s source code and picks spots that often have bugs. The second part writes small test programs aimed at those spots. The third part runs each program on BOOM and on the reference models and compares what they commit. The fourth part looks at each difference and decides what it means: a real BOOM bug, a bug in a reference model, a difference the rules allow, or a problem with the test setup. We use two reference models, Spike and Dromajo, not one. With two, the tool can often decide by majority vote instead of guessing. To measure how well it works, we planted known bugs in BOOM’s source and checked how many the tool finds. It found every planted bug that changes what the CPU commits, never reported a bug on the clean version, and—when we seeded a bug into one reference model—the vote pinpointed the faulty model, which a single-reference flow cannot do. We are clear about what we built and what already existed in the Berkeley tools.

Index Terms—CPU verification, RISC-V, BOOM, differential testing, co-simulation, CHIA, hardware/software co-design

I. INTRODUCTION

Comparing a real CPU design against a simple reference model is a common way to find CPU bugs. The Chipyard tools [1] already let you run Spike next to a BOOM simulation and compare them. So running the comparison is not the hard part. The hard part is deciding what to test, writing tests that reach the tricky cases, and, most of all, deciding what a difference means. When BOOM and a reference model disagree, it can be a bug in BOOM, a bug in the reference model, a difference the rules allow, or just a problem with how we ran the test.

This work builds a loop around that last problem. We are honest about credit: running Spike next to BOOM is an existing Berkeley feature, and we use it as is. What we add is the layer on top: reading the RTL to choose where to test, writing tests aimed there, and deciding what each difference means with a strong rule against calling something a BOOM bug when we are not sure.

II. BACKGROUND

CHIA [2] is an open framework for AI-driven hardware and software co-design flows. It already supports many tools,

including Chipyard, Spike [3], Verilator [4], and gem5 [5]. **BOOM** [6] is an out-of-order RISC-V [7] core; we test its third generation, SonicBOOM [8], which is written in the Chisel hardware language [9]. A **reference model** runs instructions one at a time and computes the correct result directly. An out-of-order core does the same work in a more complex way, with renaming, guessing ahead, queues, and forwarding. Those are the places where it can go wrong. Because a reference model has no idea of clock cycles, we only compare *committed state*: the list of instructions that retired, their bits, and the register values they wrote. Cycle counts and issue order are not compared.

III. RELATED WORK

The closest work is CPU fuzzing. RFUZZ [10] fuzzes RTL on FPGAs. DIFUZZRTL [11] adds register-coverage guidance and differential testing. TheHuzz [12] fuzzes at the instruction level against a golden model. ProcessorFuzz [13] guides the search using changes in control and status registers and tests Rocket, BOOM, and BlackParrot. Cascade [14] writes long, tangled programs that stop running when they hit a bug. Most recently, SimFuzz [15] uses similarity-guided mutation and differential testing to find 14 previously unknown bugs (7 CVEs) in Rocket, BOOM, and XiangShan. Constrained-random generators like riscv-dv [16] also make legal test programs. These fuzzers are strong at *finding* bugs: they run enormous numbers of inputs and, like SimFuzz, discover real, previously unknown bugs. We do not compete on raw discovery or throughput. Instead we focus on a different, less studied question: once a difference appears, *deciding what it means*—a BOOM bug, a reference-model bug, an allowed difference, or an artifact. To that end we read the RTL to aim tests and, more importantly, use two reference models to classify each difference by a mechanical vote. This matters: ProcessorFuzz found a bug in a reference model itself, which shows a difference is not always a bug in the design. The classic idea of comparing two implementations of the same rules comes from compiler testing, such as Csmith [17]. Separately, large language models are now used to write and check hardware: ChipNeMo [18] adapts a model to chip-design tasks, and VerilogEval [19] benchmarks LLMs on Verilog generation. Our work uses a model to choose and write tests, not to write the design.

IV. HOW THE LOOP WORKS

The loop has four parts. Each is a separate CHIA block.

A. Pick spots

This part reads BOOM’s source and looks for spots that often hold bugs, in five groups: exception priority, queue pointers, forwarding and bypass, memory ordering, and code the authors marked as unfinished (TODO and disabled features). Each hit is saved with a `file:line` pointer, so every later test can be traced back to the exact line that motivated it. On BOOM v3 (56 Scala files, about 18.8k lines) the scan finds 423 spots. A model then turns the top spots into test ideas, each with a setup, a test sketch, and the exact difference it expects to see. We rank the spots so a memory-ordering idea points at the load/store unit, not at an unrelated line that only matched the same keyword.

B. Write tests

This part turns each idea into a small bare-metal RISC-V program. It wraps the program with a trap handler and a clean exit. The key part is a check: a test is only kept if it assembles, finishes, and runs enough instructions to do real work. A program that runs nothing produces a clean comparison and proves nothing, so a loop that counts clean runs as success would happily report that a broken test writer found no bugs. Rejected tests are counted and reported. They mean the check is working, not that the loop failed.

C. Run and compare

This part turns both models’ output into one common list of committed instructions and compares them step by step from a start point, stopping at the first difference. It starts from the program’s entry, not from reset, because the two models boot differently. A useful practical note: BOOM’s commit log uses almost the same format as Spike, so one parser reads both.

D. Decide what it means

This part sorts each difference into one of five kinds: a BOOM bug, a reference-model bug, an allowed difference, a test-setup problem, or not sure. Easy cases are decided first with no model call. A difference that does not repeat across runs of two fixed models is not sure. A difference on a free-running counter (`cycle`, `time`, `instret`) is allowed by the rules. An empty or cut-off trace is a test-setup problem, and we tell the two apart because saying “raise the instruction budget” is useless when the simulator never started. Only what is left goes to the model, and with real evidence: the disassembly at both program counters, nearby trace, and the RTL line the idea pointed at. The bar for calling something a BOOM bug is high on purpose. A wrong bug report to BOOM’s authors costs more than a missed one, so anything the evidence does not settle is marked “not sure” instead of being rounded up. Only real BOOM-bug and reference-bug results feed back into the next round of ideas.

E. Vote with two reference models

Older CPU fuzzers use one reference model, so a difference leaves “which side is wrong?” to a person. We add a second reference model, Dromajo [20], next to Spike, and let the tool vote. Call the implementation B and the two references A_1 and A_2 . If all three agree, it is clean. If both references agree and B is the odd one out, it is a BOOM bug. If B matches one reference but not the other, the odd reference is the suspect. If the two references disagree with each other, that is a reference-model problem, apart from BOOM. We call this triangulation. It turns the hard, human step into a simple vote for most cases, and only sends a true three-way split to the model. If a bug makes BOOM run forever, a cycle cap stops it so we still get a full, comparable trace instead of a timeout with no output.

V. IMPLEMENTATION

The loop is about 3.8k lines of Python with no extra runtime libraries. It drives a toolchain we script fully so it rebuilds the same way on a fresh cloud account: Spike (pinned to one commit, built from source), `riscv64-unknown-elf-gcc` 14.2, Verilator 5.022, Dromajo, and a Chipyard BOOM v3 Verilator model with commit logging on. The whole toolchain installs without root. The model calls use Gemini and fall back to a fixed offline path when no key is set, so the loop still runs and can be tested at no cost. Every result records where it came from: a fixed rule, the offline path, or which model. The code ships with 58 unit tests, most of them checks that the test gate and the decision rules reject what they should.

VI. RESULTS

All numbers below come from runs on the funded compute: BOOM built and run as a Chipyard `MediumBoomV3` Verilator model, with Spike and Dromajo as the two reference models.

A. Planted-bug study

Real bugs in a mature core are rare, so we test the loop against planted bugs. Each planted bug is one small, reversible change to BOOM’s source at a chosen spot, anchored by an exact text match so a wrong match is caught before a build, not after. The set has eleven planted bugs: three in load/store memory ordering and eight in forwarding and the datapath. Each one is a known bug, so the test gives two numbers: *recall*, how many the loop finds, and *precision*, how many of its BOOM-bug calls land on a real planted bug. A third number comes for free: any BOOM-bug call on the clean version is a false alarm and must be zero.

Two things keep the numbers honest. We only count recall over bugs that are *observable*: a bug that never changes committed state cannot be found by any comparison, so we check each missed bug against the clean baseline and mark it “not observable” instead of counting it as a miss. And we check that each planted binary really differs from the baseline, so a “not observable” result can never come from a cached or failed build.

Results. Of the eleven planted bugs (Table I), eight change committed state under our tests and three do not. The vote

Planted bug	Site	Result
BYPASS_RS1_DISABLED	register-read	found
BYPASS_RS2_DISABLED	register-read	found
BYPASS_RS1_MATCH_WRONG	register-read	found
BYPASS_RS2_MATCH_WRONG	register-read	found
BYPASS_RS1_ZERO_GUARD	register-read	found
BYPASS_RS2_ZERO_GUARD	register-read	found
FORWARD_STD_VAL_STUCK	lsu	found
FWD_IRESP_EARLY	lsu	found
ORDER_FAIL_SUPPRESSED	lsu	not observable
FENCEI_EARLY	lsu	not observable
MOV_WRONG_OPERAND	exec unit	not observable

TABLE I

PLANTED-BUG RESULTS. EVERY BUG THAT CHANGES COMMITTED STATE ON A SINGLE CORE WAS FOUND; THE THREE THAT NEVER CHANGE COMMITTED STATE ARE LEFT OUT OF RECALL, NOT COUNTED AS MISSES.

Spike (reference)	BOOM
0x10 mv a0, s2 (handler entry)	0x10 mv a0, s2 0x12 ecall ← logged
0x50 csrr mcause	0x50 csrr mcause

Fig. 1. Finding 1. BOOM logs the trapping `ecall`; Spike does not. Both reach the same handler with the same `mcause`. The apparent control-flow difference is a logging habit, not a bug.

found all eight that do: recall over observable bugs is **8 of 8**. Precision is **100%**: on the clean baseline the tool made *zero* BOOM-bug calls, so every call landed on a real planted bug. The three not-observable bugs are two multi-core memory-ordering effects and one register-move case our tests never trigger. These are honest limits of single-core bare-metal testing, not tool failures. Each one builds a binary that really differs from the baseline yet commits the same state.

Finding 1: a logging habit that looks like a bug. Our exception test traps on purpose. The comparison first reported a control-flow difference. We looked instead of assuming. BOOM logs the trapping instruction (for example `ecall`) with no register write, while Spike jumps straight to the handler. Both then enter the same handler with the same `mcause`, so the state is the same. It is a logging habit, not a bug. Left alone it would raise a false alarm on every exception test. We handle it, but we *count and report* each skip instead of quietly realigning, since a tool that quietly realigns can hide a real bug. Handling it showed a second case, the compressed `c.ebreak` form, that our first list missed. The same habit shows up *between the two references*: on the exception test Spike and Dromajo disagree, which the vote flags as a reference-model mismatch. A closer look shows it is the same logging habit, not a reference-model bug, and the tool does not report it as one.

Finding 2: the second reference finds a bug the first misses. The `FORWARD_STD_VAL_STUCK` bug breaks store-to-load forwarding. The wrong value throws the program off course, so the difference shows up as a cut-off trace, which single-reference triage calls a test-setup problem and misses. The vote finds it: both references finish and agree, BOOM is the odd one out, so the call is a BOOM bug no matter how the difference looks. This is the clear case for the second reference: a bug the one-reference path missed, found by a simple vote.

Finding 3: repeat and evidence checks matter. Two bugs

in our own harness, including a wrong file path that gave an empty BOOM trace, were caught by the decision part, which correctly refused to call an empty trace a bug. This is why we run fixed checks before any model call: repeat first, and tell “the tool did not run” apart from “the models disagree.”

Finding 4: a real-input probe, correctly attributed. Beyond planted bugs, we pointed the loop at a documented BOOM issue (a report that misaligned loads could return corrupted data instead of trapping). On our build the bug does not reproduce: BOOM traps the misaligned load and *agrees with Spike*. The loop did not blame BOOM. It named the outlier reference instead: BOOM and Spike trap, while Dromajo permits misaligned loads, so the vote returns `MODEL_DEFECT` pointing at Dromajo. This is a policy difference between the references (misaligned support), not necessarily a Dromajo bug—but the loop correctly kept BOOM clear and identified the source of the difference.

Finding 5: catching a bug in a reference model. To test the two-oracle capability directly, we seeded a bug into Spike itself—`MUL` off by one—and ran a multiply test. BOOM and Dromajo agree on the correct product; buggy Spike is the sole outlier. The vote returns `MODEL_DEFECT` and names *Spike*, correctly identifying which reference model is wrong. With a correct Spike the same test is `CLEAN`. A single-golden-model flow cannot make this call: it would see a mismatch and could not tell whether the CPU or the reference was at fault. This is the concrete payoff of a second reference—and, since reference models do carry real bugs (ProcessorFuzz found one), a capability that matters in practice.

B. Random-program campaigns

Targeted tests aim at spots we picked. To cover ground we did not pick, we also ran many constrained-random programs, in the spirit of SimFuzz [15] and DIFUZZRTL [11] but simpler: each program is legal, terminating, and trap-free, so any difference is a real disagreement and not a crash we caused.

Integer campaign. We generated 200 random RV64IM programs—straight-line register, immediate, shift, multiply/divide, and aligned load/store instructions over a fixed scratch buffer—and ran each through the vote against BOOM and both references. Every one came back clean: BOOM matched both references on all committed state, across all 200 programs.

Floating-point campaign. We then turned on the FPU (RV64IMFD) and wrote a second generator aimed at the parts of floating point that implementations get wrong most often: NaN-boxing of single-precision values, `min/max` with a NaN operand, saturating float-to-integer conversion, signed zero, fused multiply-add, and all five rounding modes. One design choice keeps the vote sound: the three models write floating-point results to their logs in different formats, so we never compare the float registers directly. Instead each result is moved into an integer register (`fmv.x.d`) and the accumulated exception flags are read into an integer register (`csrr fflags`), so every observable value lands in the integer file, which all three models report the same way. We ran 80 such programs,

about 8,000 floating-point instructions in all. Every one came back clean.

Two false alarms, caught. On the way there the vote twice called BOOM the outlier. Both times we did what Finding 1 teaches: read the exact instruction before calling it a bug. Both were our own test setup, not BOOM. The first was the exception-flag register: BOOM starts it with bits set that the references start clear, so a test that never cleared it was comparing leftover reset state. The second was the rounding mode: a dynamically-rounded subtract differed by one unit in the last place because we had not set the rounding mode, and BOOM’s reset rounding mode differs from the references’. Clearing the whole floating-point control register first—both the flags and the rounding mode—made all three agree. The lesson is the one the loop is built on: a vote proposes, the exact instruction disposes, and controlling the machine’s start state is part of an honest floating-point test.

What the campaigns show. We did not find a new bug in BOOM. What we did show is that BOOM’s integer and floating-point datapaths agree with two independent references across many random inputs, and that the triage keeps false alarms out even when the start state is tricky. The strength on show here is detection and triage, not discovery on this harness.

VII. LIMITATIONS AND FUTURE WORK

We are explicit about four limits.

First, single-core bare-metal tests cannot see every kind of bug: three of our eleven planted bugs never change committed state this way, so no comparison can catch them. Two are multi-core memory effects that need more than one hart. This is also why the documented open bugs we looked at do not reproduce here: they need virtual memory, a change of privilege, or more than one core to reach, and our harness sets up none of these. The gap is not detection but reach—the vote finds a difference wherever the test can make one appear, but a bare-metal single-core program cannot drive the machine into those states in the first place. A richer harness is the honest way to close it (below).

Second, our results do not isolate the value of aiming tests at specific RTL spots. Seven of the eight caught bugs were detected by one broad, mixed program rather than by the targeted programs, and because the study stops at the first detection, the targeted programs often did not run. So the loop finds these bugs by comparing and voting on a datapath-exercising program; whether reading the RTL to aim tests actually improves detection is not shown here and needs a controlled comparison. The eight observable bugs are also close relatives (bypass and forwarding), so the recall number covers a narrow part of the microarchitecture.

Third, we have not seen a reference model fail on its own in the wild: on real programs the two references agreed on all committed state except a misaligned-load policy difference (Finding 4). We therefore demonstrate the tie-breaking capability with a *seeded* reference-model bug (Finding 5), not a

naturally occurring one; finding a real reference-model bug is future work.

Fourth, these are planted bugs that show the method works; we have not yet found a new, real bug in BOOM.

VIII. IDEAS TO EXTEND THE LOOP

Three more steps would make the loop stronger. Each builds on existing work.

A formal reference model. Adding the Sail formal RISC-V model [21], the official executable spec, as a third reference would make the vote even stronger, since it is grounded in the written rules.

Coverage feedback for choosing spots. Right now the loop reads only static source. Feeding it live signals, such as CSR changes like ProcessorFuzz [13] or register coverage like DIFUZZRTL [11], would tell the model where the tests have not been yet. No current fuzzer uses a model to choose where to aim.

Deeper test programs. Straight-line programs cannot reach every state. Cascade’s [14] tangled programs reach much deeper, which would raise how many planted bugs are observable and lift the ceiling on recall.

A richer harness. Most documented BOOM bugs live behind virtual memory, a change of privilege, or a second core. A harness that sets up page tables, steps through the privilege levels, and boots more than one hart would let the same vote reach those bug classes. This is the single change most likely to turn the method from catching planted bugs into finding real ones.

Formal check of a difference. A difference found by testing is a cheap lead; a bounded formal check is a proof. A found difference could seed a formal property, checked by the yosys/SymbiYosys flow we built earlier, to turn a suspect trace into a proof or a small counterexample.

IX. CONCLUSION

We built a full, repeatable loop that looks for bugs in BOOM by comparing it to two reference models. We credit the co-simulation as a Berkeley feature and add the layer on top: reading the RTL to choose tests, writing tests that do real work, and deciding what each difference means. The tool is careful not to invent findings: a gate that drops empty tests, fixed rules that settle the easy cases, and a high bar for calling something a BOOM bug. On planted bugs it found every one that a single core can show, made no false alarms, and, with a second reference model, found a bug the one-reference path missed.

REFERENCES

- [1] A. Amid, D. Biancolin, A. Gonzalez *et al.*, “Chipyard: Integrated design, simulation, and implementation framework for custom SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [2] SLICE Lab, UC Berkeley, “CHIA: Composable AI-driven hardware/software co-design flows,” <https://chialoops.ai>, 2025.
- [3] RISC-V Software Contributors, “Spike, the RISC-V ISA simulator,” <https://github.com/riscv-software-src/riscv-isa-sim>, 2024.
- [4] W. Snyder, “Verilator: Open-source SystemVerilog simulator,” <https://www.veripool.org/verilator/>, 2024.

- [5] N. Binkert, B. Beckmann, G. Black *et al.*, “The gem5 simulator,” *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.
- [6] C. Celio, D. A. Patterson, and K. Asanović, “The Berkeley out-of-order machine (BOOM): An industry-competitive, synthesizable, parameterized RISC-V processor,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167, 2015.
- [7] A. Waterman and K. Asanović, “The RISC-V instruction set manual, volume I: Unprivileged ISA,” 2019, rISC-V Foundation.
- [8] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanović, “SonicBOOM: The 3rd generation Berkeley out-of-order machine,” in *Fourth Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2020.
- [9] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzynek, and K. Asanović, “Chisel: Constructing hardware in a Scala embedded language,” in *49th Annual Design Automation Conference (DAC)*, 2012.
- [10] K. Laeuffer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, “RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018.
- [11] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, “DIFUZZRTL: Differential fuzz testing to find CPU bugs,” in *IEEE Symposium on Security and Privacy (S&P)*, 2021.
- [12] R. Kande, A. Crump, G. Persyn, P. Jauernig, A.-R. Sadeghi, A. Tyagi, and J. Rajendran, “TheHuzz: Instruction fuzzing of processors using golden-reference models for finding software-exploitable vulnerabilities,” in *USENIX Security Symposium*, 2022.
- [13] S. Canakci, C. Rajapaksha, L. Delshadtehrani, A. Nataraja, M. B. Taylor, M. Egele, and A. Joshi, “ProcessorFuzz: Guiding processor fuzzing using control and status registers,” *arXiv preprint arXiv:2209.01789*, 2022.
- [14] F. Solt, K. Ceesay-Seitz, and K. Razavi, “Cascade: CPU fuzzing via intricate program generation,” in *USENIX Security Symposium*, 2024.
- [15] H. Lyu, J. Wu, X. Ling, Y. Zhong, Z. Li, and T. Luo, “SimFuzz: Similarity-guided block-level mutation for RISC-V processor fuzzing,” *arXiv preprint arXiv:2601.11838*, 2026.
- [16] CHIPS Alliance, “riscv-dv: SV/UVM-based RISC-V instruction generator,” <https://github.com/chipsalliance/riscv-dv>, 2023.
- [17] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.
- [18] M. Liu *et al.*, “ChipNeMo: Domain-adapted LLMs for chip design,” *arXiv preprint*, 2023.
- [19] M. Liu, N. Pinckney, B. Khailany, and H. Ren, “VerilogEval: Evaluating large language models for Verilog code generation,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [20] Esperanto Technologies, “Dromajo: A RISC-V RV64GC reference model,” <https://github.com/chipsalliance/dromajo>, 2023.
- [21] A. Armstrong, T. Bauereiss, B. Campbell, A. Reid, K. E. Gray, R. M. Norton, P. Mundkur, M. Wassell, J. French, C. Pulte, S. Flur, I. Stark, N. Krishnaswami, and P. Sewell, “ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS,” *Proceedings of the ACM on Programming Languages (POPL)*, vol. 3, 2019.